

FLOWDAR

Détection automatique des inondations et guidage citoyen à Douala

Document technique — Spécifications Frontend

Angular Talent Lab 2026 — Orange Digital Center Douala

1. Stack technique

Couche	Technologie	Usage
Framework	Angular 21+ (standalone components)	Structure de toute l'application, routing, services
Styles	TailwindCSS	UI responsive, mobile-first, classes utilitaires
Carte	Google Maps JavaScript API	Affichage carte, marqueurs, itinéraires
Directions	Google Maps Directions API	Calcul d'itinéraires contournant les zones alertées
Autocomplete	Google Maps Places API	Champ de recherche de destination (Ecran 5)
Temps réel	Firebase Firestore (onSnapshot)	Mise à jour automatique des alertes sans rechargement
Auth	Firebase Auth	Connexion email/password + Google Sign-In
Fichiers	Firebase Storage	Upload et stockage des photos de signalement
HTTP	Angular HttpClient	Appels vers l'API REST du backend Node.js
Offline	Firestore persistence + localStorage	Consultation hors connexion des dernières alertes
Déploiement	Firebase Hosting	Mise en ligne gratuite avec CDN mondial

Packages npm à installer

```
npm install @angular/google-maps # Google Maps natif Angular
npm install @angular/fire # Firebase (Auth + Firestore + Storage)
```

2. Structure des dossiers

Chaque dossier a un rôle précis — core (cerveau), features (pages), shared (composants réutilisables).

```
src/app/
├── core/
│   ├── services/
│   │   ├── alerte.service.ts      → CRUD alertes (API REST + Firestore)
│   │   └── auth.service.ts        → Firebase Auth (connexion, déconnexion,
état)
│   ├── maps.service.ts           → Google Maps + calcul itinéraires
│   ├── weather.service.ts        → Données météo depuis le backend
│   └── guards/
│       ├── auth.guard.ts         → Redirige vers /auth si non connecté
│       └── no-auth.guard.ts      → Redirige vers / si déjà connecté
├── features/
│   ├── carte/
│   │   └── carte.component.ts      → Composant principal (page)
│   │   ├── carte-marqueur.component.ts → Marqueur coloré sur la carte
│   │   └── carte-bottomsheet.component.ts → Popup résumé au clic marqueur
│   ├── alerte-detail/
│   │   ├── alerte-detail.component.ts
│   │   └── confirmation-list.component.ts → Liste des confirmations citoyennes
│   ├── alertes-preventives/
│   │   └── alertes-preventives.component.ts
│   ├── signalement/
│   │   ├── signalement.component.ts → Stepper 3 étapes
│   │   ├── step-zone.component.ts   → Etape 1 : quartier + niveau
│   │   ├── step-details.component.ts → Etape 2 : description + photo
│   │   └── step-recap.component.ts   → Etape 3 : récapitulatif + envoi
│   ├── itineraire/
│   │   ├── itineraire.component.ts
│   │   └── itineraire-carte.component.ts → Carte avec itinéraire tracé
│   ├── historique/
│   │   └── historique.component.ts
│   └── auth/
│       ├── auth.component.ts      → Ecran 7 : login / inscription
│       ├── login.component.ts     → Toggle login/inscription
│       └── register.component.ts
└── shared/
    └── components/
```

		navbar/	→ Barre navigation bas (présente partout)
		alerte-card/	→ Carte alerte réutilisable (liste +
historique)		badge-niveau/	→ Badge coloré Léger/Moyen/Dangereux
		badge-statut/	→ Badge Actif/Résolu/Préventif
		meteo-bandeau/	→ Bandeau météo actuelle Douala
		pipes/	
		temps-ecoule.pipe.ts	→ 'il y a 2h' depuis un timestamp

3. Routes et guards

Route	Composant	Accès	Guard
/	CarteComponent	Tout le monde	Aucun
/alerte/:id	AlerteDetailComponent	Tout le monde	Aucun
/preventives	AlertesPreventivesCompon ent	Tout le monde	Aucun
/itineraire	ItineraireComponent	Tout le monde	Aucun
/historique	HistoriqueComponent	Tout le monde	Aucun
/signaler	SignalementComponent	Connecté uniquement	AuthGuard
/auth	AuthComponent	Non connecté uniquement	NoAuthGuard

4. Détail des 7 écrans

Ecran 1 — Carte principale (page d'accueil)

C'est l'écran le plus important — celui que l'utilisateur voit en premier dès l'ouverture de l'app.

Contenu de l'écran :

- Carte Google Maps plein écran
- Marqueurs colorés sur les zones alertées : jaune (léger) / orange (moyen) / rouge (dangereux)
- Marqueur bleu = position actuelle de l'utilisateur (API Geolocation)
- Bandeau en haut : météo actuelle Douala (icône pluie + mm/h depuis le backend)
- Bouton flottant (FAB) 'Signaler une zone' en bas à droite
- Bottom navigation bar : Carte / Preventives / Itinéraire / Historique / Profil

- Mise à jour automatique via Firestore onSnapshot — aucun rechargement nécessaire

Comportements :

- Clic sur un marqueur → bottom sheet qui s'ouvre avec résumé rapide (quartier, niveau, heure, nb confirmations)
- Bouton 'Voir le détail' dans le bottom sheet → navigation vers Ecran 2
- Si pas de réseau → affichage des alertes en cache avec bandeau 'données hors ligne'

Ecran 2 — Détail d'une alerte

Contenu de l'écran :

- Nom du quartier + badge niveau coloré
- Source : 'Détection automatique', 'Signalement citoyen' ou 'ONACC officiel'
- Heure de début + temps écoulé calculé par le pipe temps-ecoule ('il y a 2h')
- Nombre de confirmations citoyennes avec icône
- Résumé Gemini : texte lisible généré par l'IA (ex: 'Rue Mandessi Bell sous 80cm d'eau')
- Photo si disponible (chargée depuis Firebase Storage)
- Bouton 'Confirmer — c'est encore là' (citoyen connecté uniquement)
- Bouton 'C'est passé' (citoyen connecté uniquement)
- Section 'Confirmations' : liste des confirmations avec heure

Comportements :

- Si utilisateur non connecté et clique sur Confirmer → redirection vers /auth
- Après confirmation réussie → nb_confirmations mis à jour en temps réel via Firestore

Ecran 3 — Alertes préventives

Contenu de l'écran :

- Bandeau météo : prévisions pluie heure par heure pour Douala
- Liste des zones à risque dans les 6 prochaines heures
- Triées par heure prévue (la plus proche en premier)
- Chaque carte affiche : quartier + heure prévue + niveau de risque prévu + probabilité en %

Ecran 4 — Signalement citoyen (3 étapes)

Accessible uniquement aux utilisateurs connectés — AuthGuard actif sur cette route.

Etape 1 — Zone et niveau :

- Liste déroulante des quartiers de Douala (chargée depuis /api/zones-a-risque)
- Sélecteur de niveau : Léger / Moyen / Dangereux (3 boutons visuels colorés)

Etape 2 — Détails :

- Textarea : description libre de la situation
- Upload photo optionnel (compressée avant upload Firebase Storage)

Etape 3 — Récapitulatif :

- Résumé du signalement avant envoi
- Bouton 'Envoyer le signalement'
- Feedback après envoi : 'Signalement reçu, en cours de validation par la communauté'

Ecran 5 — Itinéraire sûr

Contenu de l'écran :

- Champ 'Ma destination' avec autocomplete Google Places
- Point de départ = position actuelle de l'utilisateur (ou saisie manuelle)
- Bouton 'Calculer l'itinéraire sûr'
- Carte Google Maps avec l'itinéraire tracé en bleu
- Zones alertées visibles sur la carte en superposition (marqueurs rouges)
- Bandeau d'information : 'Itinéraire modifié — évite Ndokotti (inondé) et Bépanda (risque à 17h)'
- Si aucune zone alertée sur le trajet : bandeau vert 'Votre itinéraire est praticable'

Comportement technique :

- Angular appelle le backend : POST /api/itineraire avec destination
- Backend retourne les coordonnées GPS des zones actives à éviter
- Angular passe ces coordonnées à Google Maps Directions API
- Google Maps calcule et affiche l'itinéraire contournant ces zones

Ecran 6 — Historique

Contenu de l'écran :

- Filtre par quartier (liste déroulante)
- Liste des alertes passées : date, durée totale, niveau, nombre de confirmations
- Statistique par quartier : 'Ndokotti — 12 alertes ce mois-ci'

Ecran 7 — Connexion / Inscription

Contenu de l'écran :

- Logo Flowdar centré en haut
- Toggle Login / Inscription
- Champs : email + mot de passe
- Bouton Google Sign-In (Firebase Auth)
- Champ quartier de domicile (inscription uniquement — utilisé pour le filtrage des alertes)
- Redirection automatique vers la carte après connexion réussie

5. Services Angular — Détail

alerte.service.ts

```
// Méthodes à implémenter :

getAlertesActives()           // GET /api/alertes → Observable via Firestore
onSnapshot
getAlerteById(id)             // GET /api/alertes/:id
getAlertesPreventives()       // GET /api/alertes?type=préventive
getHistorique(quartier)       // GET /api/alertes/historique/:quartier
confirmerAlerte(id)           // POST /api/alertes/:id/confirmer
resoudreAlerte(id)            // POST /api/alertes/:id/resoudre
signalerZone(signalement)     // POST /api/alertes/signaler
```

auth.service.ts

```
// Méthodes à implémenter :

connexionEmail(email, password) // Firebase signInWithEmailAndPassword
connexionGoogle()               // Firebase signInWithPopup (GoogleAuthProvider)
inscription(email, password, quartier) // Firebase createUserWithEmailAndPassword
deconnexion()                   // Firebase signOut
getUtilisateurActuel()          // Observable<User | null> via authState
estConnecte()                   // boolean — utilisé par les guards
```

maps.service.ts

```
// Méthodes à implémenter :

initialiserCarte(element)       // Créer l'instance Google Maps
ajouterMarqueur(alerte)         // Ajouter un marqueur coloré selon le niveau
supprimerMarqueurs()            // Nettoyer la carte avant refresh
calculerItineraire(origine, dest, zonesAEviter) // Directions API
```

```
getPositionActuelle() // Promise<{lat, lng}> via Geolocation API
```

weather.service.ts

```
// Méthodes à implémenter :  
  
getMeteoActuelle() // GET /api/meteo/actuelle → pluie actuelle mm/h  
getPrevisions() // GET /api/meteo/previsions → prévisions 6h
```

6. Variables d'environnement Angular

```
// src/environments/environment.ts  
export const environment = {  
  production: false,  
  backendUrl: 'http://localhost:3000', // URL backend Node.js  
  googleMapsApiKey: '...', // Clé Google Maps  
  firebaseConfig: {  
    apiKey: '...',  
    authDomain: '...',  
    projectId: '...',  
    storageBucket: '...',  
    messagingSenderId: '...',  
    appId: '...'  
  }  
};
```

7. Comportements hors connexion

Situation	Comportement Angular
Pas de réseau au démarrage	Alertes du cache localStorage affichées + bandeau 'Données hors ligne - dernière mise à jour à HH:MM'
Réseau coupé en cours	Firestore offline persistence prend le relais automatiquement, aucune action requise
Signalement sans réseau	Signalement mis en file d'attente localStorage, envoyé automatiquement à la reconnexion
Carte Google Maps sans réseau	Carte non disponible — message 'Carte indisponible hors ligne, liste des alertes disponible'
Retour en ligne	Firestore se resynchronise automatiquement, cache mis à jour

8. Communication avec le backend

Angular communique avec le backend Node.js via HttpClient sur tous les endpoints suivants :

Endpoint	Méthode	Utilisé dans	Données envoyées
/api/alertes	GET	alerte.service / Ecran 1	Aucune
/api/alertes/:id	GET	alerte.service / Ecran 2	Aucune
/api/alertes/:id/confirmer	POST	Ecran 2 (bouton Confirmer)	{ user_uid }
/api/alertes/:id/resoudre	POST	Ecran 2 (bouton C'est passé)	{ user_uid }
/api/alertes/signaler	POST	Ecran 4 (formulaire)	{ quartier, niveau, description, photo_url }
/api/alertes/historique/:quartier	GET	Ecran 6	Aucune
/api/zones-a-risque	GET	Ecran 4 (liste déroulante)	Aucune
/api/itineraire	POST	Ecran 5	{ destination_lat, destination_lng }
/api/meteo/actuelle	GET	weather.service / Ecran 1	Aucune
/api/meteo/previsions	GET	weather.service / Ecran 3	Aucune

Firestore onSnapshot est utilisé uniquement pour les alertes actives (Ecran 1) afin d'avoir la carte mise à jour en temps réel. Tous les autres appels passent par l'API REST du backend.

9. Checklist de livraison frontend

1. Projet Angular 21+ initialisé (ng new flowdar --standalone)
2. TailwindCSS configuré
3. Firebase configuré (Auth + Firestore offline persistence + Storage)
4. Google Maps API intégrée (@angular/google-maps)
5. Routing configuré avec les 7 routes et les guards
6. Les 3 services core implémentés (alerte, auth, maps, weather)
7. Les 7 écrans développés avec leurs composants standalone

8. Composants shared implémentés (navbar, alerte-card, badge-niveau, badge-statut, meteo-bandeau)
9. Pipe temps-ecoule fonctionnel
10. Comportement offline testé (cache localStorage + Firestore persistence)
11. Variables d'environnement configurées (development + production)
12. Déploiement Firebase Hosting opérationnel